
pampaneira_imputation

Release 0.1.0

Ricardo Ruiz

Apr 07, 2025

CONTENTS:

1	pampaneira_imputation package	3
1.1	Submodules	3
1.2	pampaneira_imputation.config module	3
1.3	pampaneira_imputation.data_loader module	3
1.4	pampaneira_imputation.data_preprocessor module	5
1.5	pampaneira_imputation.evaluation module	9
1.6	pampaneira_imputation.imputation_methods module	9
1.7	pampaneira_imputation.utils module	11
1.8	Module contents	12
2	tests package	13
2.1	Submodules	13
2.2	tests.conftest module	13
2.3	tests.minimal_impute_function module	13
2.4	tests.minimal_test module	13
2.5	tests.test_data_preprocessor module	13
2.6	tests.test_evaluation module	13
2.7	tests.test_function_standalone module	13
2.8	tests.test_imputation_methods module	13
2.9	tests.test_utils module	13
2.10	Module contents	13
	Python Module Index	15
	Index	17

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

PAMPANEIRA_IMPUTATION PACKAGE

1.1 Submodules

1.2 pampaneira_imputation.config module

1.3 pampaneira_imputation.data_loader module

```
pampaneira_imputation.data_loader.load_intersection_data(filepath: str =  
                                                         './data/trafico_contamina_intersección.csv',  
                                                         date_col_original: str = 'Date',  
                                                         date_col_target: str = 'date',  
                                                         truck_pos_col: str = 'truck_pos',  
                                                         target_truck_pos: str = 'PAM_2',  
                                                         timezone: str = 'UTC') → DataFrame
```

Carga datos de intersección, filtra por posición de camión, convierte la fecha.

Args:

filepath (str, optional): Ruta al archivo CSV de intersección (por defecto: config.INTERSECTION_FILE).
date_col_original (str, optional): Nombre original de la columna de fecha en el CSV (por defecto: "Date").
date_col_target (str, optional): Nombre objetivo de la columna de fecha (por defecto: config.DATE_COL).
truck_pos_col (str, optional): Nombre de la columna de posición del camión (por defecto: config.TRUCK_POS_COL).
target_truck_pos (str, optional): Posición objetivo del camión para filtrar (por defecto: config.TARGET_TRUCK_POS).
timezone (str, optional): Zona horaria para las fechas (por defecto: config.TIMEZONE).

Returns:

pd.DataFrame: DataFrame con datos de intersección cargados, filtrados y preprocesados.

Raises:

FileNotFoundError: Si el archivo especificado no se encuentra. KeyError: Si una columna especificada no se encuentra o falla el cambio de nombre en el archivo.

```
pampaneira_imputation.data_loader.load_traffic_data(filepath: str = '../data/trafico_feb22_ago23.csv',
                                                    columns_to_use: List[str] =
                                                    ['vehicles_PAM_1_OUT',
                                                     'vehicles_PAM_1_OUT_Zona_Granada', 'vehic-
                                                     cles_PAM_1_OUT_Zona_Catalunia_y_Otras',
                                                     'vehi-
                                                     cles_PAM_1_OUT_Zona_Andalucia_no_GR',
                                                     'vehicles_PAM_1_OUT_Extranjero', 'vehi-
                                                     cles_PAM_1_OUT_Zona_Comunidad_de_Madrid',
                                                     'vehi-
                                                     cles_PAM_1_OUT_Zona_Extremadura_y_Otras',
                                                     'vehicles_PAM_1_OUT_Zona_Otras',
                                                     'vehicles_PAM_1_OUT_5_seats',
                                                     'vehicles_PAM_1_OUT_+5_seats',
                                                     'vehicles_PAM_1_OUT_-5_seats',
                                                     'vehicles_PAM_1_OUT_nan_seats',
                                                     'vehicles_PAM_1_OUT_101-200_CO2',
                                                     'vehicles_PAM_1_OUT_0-100_CO2',
                                                     'vehicles_PAM_1_OUT_nan_CO2',
                                                     'vehicles_PAM_1_OUT_201-300_CO2',
                                                     'vehicles_PAM_1_OUT_+300_CO2',
                                                     'vehicles_PAM_1_IN',
                                                     'vehicles_PAM_1_IN_Zona_Granada',
                                                     'vehicles_PAM_1_IN_Zona_Catalunia_y_Otras',
                                                     'vehicles_PAM_1_IN_Zona_Andalucia_no_GR',
                                                     'vehicles_PAM_1_IN_Extranjero', 'vehi-
                                                     cles_PAM_1_IN_Zona_Comunidad_de_Madrid',
                                                     'vehi-
                                                     cles_PAM_1_IN_Zona_Extremadura_y_Otras',
                                                     'vehicles_PAM_1_IN_Zona_Otras',
                                                     'vehicles_PAM_1_IN_5_seats',
                                                     'vehicles_PAM_1_IN_+5_seats',
                                                     'vehicles_PAM_1_IN_-5_seats',
                                                     'vehicles_PAM_1_IN_nan_seats',
                                                     'vehicles_PAM_1_IN_101-200_CO2',
                                                     'vehicles_PAM_1_IN_0-100_CO2',
                                                     'vehicles_PAM_1_IN_nan_CO2',
                                                     'vehicles_PAM_1_IN_201-300_CO2',
                                                     'vehicles_PAM_1_IN_+300_CO2',
                                                     'vehicles_PAM_2_OUT',
                                                     'vehicles_PAM_2_OUT_Zona_Granada', 'vehi-
                                                     cles_PAM_2_OUT_Zona_Catalunia_y_Otras',
                                                     'vehi-
                                                     cles_PAM_2_OUT_Zona_Andalucia_no_GR',
                                                     'vehicles_PAM_2_OUT_Extranjero', 'vehi-
                                                     cles_PAM_2_OUT_Zona_Comunidad_de_Madrid',
                                                     'vehi-
                                                     cles_PAM_2_OUT_Zona_Extremadura_y_Otras',
                                                     'vehicles_PAM_2_OUT_Zona_Otras',
                                                     'vehicles_PAM_2_OUT_5_seats',
                                                     'vehicles_PAM_2_OUT_+5_seats',
                                                     'vehicles_PAM_2_OUT_-5_seats',
                                                     'vehicles_PAM_2_OUT_nan_seats',
                                                     'vehicles_PAM_2_OUT_101-200_CO2',
                                                     'vehicles_PAM_2_OUT_0-100_CO2',
                                                     'vehicles_PAM_2_OUT_nan_CO2',
                                                     'vehicles_PAM_2_OUT_+300_CO2',
                                                     'vehicles_PAM_2_IN',
                                                     'vehicles_PAM_2_IN_Zona_Granada',
```


Carga datos de tráfico generales, selecciona columnas relevantes, convierte la fecha.

Args:

filepath (str, optional): Ruta al archivo CSV de tráfico (por defecto: config.TRAFFIC_FILE). columns_to_use (List[str], optional): Lista de columnas de tráfico a usar (por defecto: config.PAM_BUB_TRAFFIC_COLS). date_col (str, optional): Nombre de la columna de fecha (por defecto: config.DATE_COL). timezone (str, optional): Zona horaria para las fechas (por defecto: config.TIMEZONE).

Returns:

pd.DataFrame: DataFrame con datos de tráfico cargados y preprocesados.

Raises:

FileNotFoundError: Si el archivo especificado no se encuentra. KeyError: Si una columna especificada no se encuentra en el archivo.

1.4 pampaneira_imputation.data_preprocessor module

```
pampaneira_imputation.data_preprocessor.add_period1_padding(df_period1: DataFrame, start_pad:
Timestamp = Timestamp('2023-01-17
00:00:00+0000', tz='UTC'), end_pad:
Timestamp = Timestamp('2023-03-14
23:00:00+0000', tz='UTC'), freq: str
= 'h') → DataFrame
```

Añade relleno de NaNs al inicio y al final de los datos del Periodo 1.

Args:

df_period1 (pd.DataFrame): DataFrame del Periodo 1 con DatetimeIndex. start_pad (pd.Timestamp, optional): Fecha de inicio para el relleno inicial (por defecto: config.PERIOD_1_PADDING_START).

end_pad (pd.Timestamp, optional): Fecha de fin para el relleno final
(por defecto: config.PERIOD_1_PADDING_END).

freq (str, optional): Frecuencia para el rango de fechas de relleno
(por defecto: 'h').

Returns:

pd.DataFrame: DataFrame del Periodo 1 con relleno de NaNs añadido al
inicio y al final.

```
pampaneira_imputation.data_preprocessor.fill_missing_timestamps(df: DataFrame, start_date:
Timestamp, end_date:
Timestamp, freq: str = 'h',
date_col: str = 'date') →
DataFrame
```

Rellena las marcas de tiempo horarias faltantes en un DataFrame con NaNs.

Args:

df (pd.DataFrame): DataFrame de entrada con una columna de fecha. start_date (pd.Timestamp): Fecha de inicio del rango completo. end_date (pd.Timestamp): Fecha de fin del rango completo. freq (str, optional): Frecuencia para el rango de fechas (por defecto: 'h'). date_col (str, optional): Nombre de la columna de fecha (por defecto: config.DATE_COL).

Returns:

pd.DataFrame: DataFrame con marcas de tiempo horarias completas y NaNs para los datos faltantes.

```

pampaneira_imputation.data_preprocessor.preprocess_for_imputation(df: DataFrame, feature_cols:
list =
['vehicles_PAM_1_OUT',
'vehi-
cles_PAM_1_OUT_Zona_Granada',
'vehi-
cles_PAM_1_OUT_Zona_Catalunia_y_Otras',
'vehi-
cles_PAM_1_OUT_Zona_Andalucia_no_GR',
'vehi-
cles_PAM_1_OUT_Extranjero',
'vehi-
cles_PAM_1_OUT_Zona_Comunidad_de_Madrid',
'vehi-
cles_PAM_1_OUT_Zona_Extremadura_y_Otras',
'vehi-
cles_PAM_1_OUT_Zona_Otras',
'vehi-
cles_PAM_1_OUT_5_seats',
'vehi-
cles_PAM_1_OUT_+5_seats',
'vehicles_PAM_1_OUT_-
5_seats',
'vehi-
cles_PAM_1_OUT_nan_seats',
'vehicles_PAM_1_OUT_101-
200_CO2',
'vehicles_PAM_1_OUT_0-
100_CO2',
'vehi-
cles_PAM_1_OUT_nan_CO2',
'vehicles_PAM_1_OUT_201-
300_CO2',
'vehi-
cles_PAM_1_OUT_+300_CO2',
'vehicles_PAM_1_IN', 'vehi-
cles_PAM_1_IN_Zona_Granada',
'vehi-
cles_PAM_1_IN_Zona_Catalunia_y_Otras',
'vehi-
cles_PAM_1_IN_Zona_Andalucia_no_GR',
'vehi-
cles_PAM_1_IN_Extranjero',
'vehi-
cles_PAM_1_IN_Zona_Comunidad_de_Madrid',
'vehi-
cles_PAM_1_IN_Zona_Extremadura_y_Otras',
'vehi-
cles_PAM_1_IN_Zona_Otras',
'vehi-
cles_PAM_1_IN_5_seats',
'vehi-
cles_PAM_1_IN_+5_seats',
'vehicles_PAM_1_IN_-
5_seats',
'vehi-
cles_PAM_1_IN_nan_seats',
'vehicles_PAM_1_IN_101-
200_CO2',
'vehicles_PAM_1_IN_0-

```

Prepara los datos para modelos de imputación: divide, escala, ventanas, añade datos faltantes.

Args:

df (pd.DataFrame): DataFrame con DatetimeIndex y columnas de características. **feature_cols** (list, optional): Lista de columnas a usar como características

(por defecto: config.FEATURE_COLUMNS).

train_start (str, optional): Fecha de inicio del conjunto de entrenamiento

(por defecto: config.TRAIN_START_DATE).

train_end (str, optional): Fecha de fin del conjunto de entrenamiento

(por defecto: config.TRAIN_END_DATE).

val_start (str, optional): Fecha de inicio del conjunto de validación

(por defecto: config.VAL_START_DATE).

val_end (str, optional): Fecha de fin del conjunto de validación

(por defecto: config.VAL_END_DATE).

test_start (str, optional): Fecha de inicio del conjunto de prueba

(por defecto: config.TEST_START_DATE).

test_end (str, optional): Fecha de fin del conjunto de prueba

(por defecto: config.TEST_END_DATE).

n_steps (int, optional): Tamaño de la ventana deslizante

(por defecto: config.N_STEPS).

missing_rate (float, optional): Proporción de valores a convertir en NaN

(0 para desactivar) (por defecto: config.MISSING_RATE).

missing_pattern (str, optional): Patrón de datos faltantes: 'point', 'subseq'

o 'block' (por defecto: config.MISSING_PATTERN).

****missingness_kwargs**: Argumentos adicionales para create_missingness

(ej., min/max_length).

Returns:

Dict: Un diccionario que contiene divisiones de datos procesados

(train_X, val_X, test_X), datos originales (sin máscara) (train_X_ori, etc.), el escalador, número de características, y número de pasos. También incluye máscaras indicadoras.

```
pampaneira_imputation.data_preprocessor.split_by_period(df: DataFrame, period_1_start: Timestamp
                                                         = Timestamp('2023-01-17 17:00:00+0000',
                                                         tz='UTC'), period_1_end: Timestamp =
                                                         Timestamp('2023-03-14 11:00:00+0000',
                                                         tz='UTC'), period_2_start: Timestamp =
                                                         Timestamp('2023-06-06 13:00:00+0000',
                                                         tz='UTC'), period_2_end: Timestamp =
                                                         Timestamp('2023-06-27 00:00:00+0000',
                                                         tz='UTC'), date_col: str = 'date') →
                                                         Tuple[DataFrame, DataFrame]
```

Divide el DataFrame en dos periodos basados en fechas predefinidas.

Args:

df (pd.DataFrame): DataFrame de entrada con una columna de fecha

o DatetimeIndex.

period_1_start (pd.Timestamp, optional): Fecha de inicio del Periodo 1
(por defecto: config.PERIOD_1_START).

period_1_end (pd.Timestamp, optional): Fecha de fin del Periodo 1
(por defecto: config.PERIOD_1_END).

period_2_start (pd.Timestamp, optional): Fecha de inicio del Periodo 2
(por defecto: config.PERIOD_2_START).

period_2_end (pd.Timestamp, optional): Fecha de fin del Periodo 2
(por defecto: config.PERIOD_2_END).

date_col (str, optional): Nombre de la columna de fecha si no se usa el índice
(por defecto: config.DATE_COL).

Returns:

Tuple[pd.DataFrame, pd.DataFrame]: Tupla que contiene el DataFrame del Periodo 1 y el DataFrame del Periodo 2.

1.5 pampaneira_imputation.evaluation module

pampaneira_imputation.evaluation.calculate_imputation_metrics(*y_true: ndarray, y_pred: ndarray, indicating_mask: ndarray*) → Dict[str, float]

Calcula MAE, MSE, RMSE, MRE para valores imputados donde la máscara es 1.

Args:

y_true (np.ndarray): Datos verdaderos (potencialmente con NaNs donde faltaban originalmente). *y_pred* (np.ndarray): Datos imputados. *indicating_mask* (np.ndarray): Máscara donde 1 indica un valor faltante que fue imputado, 0 indica un valor observado.

Returns:

Dict[str, float]: Diccionario que contiene 'mae', 'mse', 'rmse', 'mre'.

pampaneira_imputation.evaluation.evaluate_all_methods(*preprocessed_data: Dict, imputed_results: Dict[str, ndarray], methods_to_evaluate: list = ['median', 'mean', 'linear', 'ffill_bfill', 'bfill_ffill', 'saits']*) → DataFrame

Evalúa múltiples métodos de imputación usando los resultados del conjunto de prueba.

Args:

preprocessed_data (Dict): Diccionario de preprocess_for_imputation. *imputed_results* (Dict[str, np.ndarray]): Diccionario que mapea nombres de métodos a arrays NumPy imputados (conjunto de prueba). *methods_to_evaluate* (list, optional): Lista de claves en imputed_results para evaluar.
(por defecto: ['median', 'mean', 'linear', 'ffill_bfill', 'bfill_ffill', 'saits'])

Returns:

pd.DataFrame: DataFrame de Pandas que resume MAE, MSE, RMSE, MRE para cada método.

1.6 pampaneira_imputation.imputation_methods module

pampaneira_imputation.imputation_methods.impute_forward_backward(*X_missing: ndarray*) → Tuple[ndarray, ndarray]

Realiza la imputación forward-fill y backward-fill en un array 3D.

Aplica forward-fill seguido de backward-fill, y backward-fill seguido de forward-fill, para imputar valores faltantes en cada muestra del array de entrada.

Args:

X_missing (np.ndarray): Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n_samples, n_steps, n_features).

Returns:

Tuple[np.ndarray, np.ndarray]: Tupla que contiene dos arrays NumPy 3D:

- El primero es el resultado de forward-fill seguido de backward-fill.
- El segundo es el resultado de backward-fill seguido de forward-fill.

`pampaneira_imputation.imputation_methods.impute_linear(X_missing: ndarray) → ndarray`

Realiza la interpolación lineal para imputar valores faltantes en un array 3D.

Aplica la interpolación lineal a lo largo del eje del tiempo (axis=0) para cada muestra en el array de entrada.

Args:

X_missing (np.ndarray): Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n_samples, n_steps, n_features).

Returns:

np.ndarray: Array NumPy 3D con valores faltantes imputados usando interpolación lineal.

`pampaneira_imputation.imputation_methods.impute_mean(X_missing: ndarray) → ndarray`

Imputa los valores faltantes en un array 3D usando la media por característica.

Calcula la media de cada característica (a través de muestras y pasos de tiempo) y reemplaza los valores NaN con la media calculada para esa característica.

Args:

X_missing (np.ndarray): Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n_samples, n_steps, n_features).

Returns:

np.ndarray: Array NumPy 3D con valores faltantes imputados.

`pampaneira_imputation.imputation_methods.impute_mean_sample_wise(data)`

Imputa los valores faltantes usando la media de cada muestra (a través del tiempo).

Si toda una característica para una muestra es NaN, usa un valor de reserva.

Args:

data (ndarray): Array NumPy de forma (n_samples, n_timestamps, n_features)

Datos de entrada con valores NaN para ser imputados.

Returns:

ndarray: Array NumPy de forma (n_samples, n_timestamps, n_features)

Datos con valores NaN imputados.

`pampaneira_imputation.imputation_methods.impute_median(X_missing: ndarray) → ndarray`

Imputa los valores faltantes en un array 3D usando la mediana por característica.

Calcula la mediana de cada característica (a través de muestras y pasos de tiempo) y reemplaza los valores NaN con la mediana calculada para esa característica.

Args:**X_missing (np.ndarray):** Array NumPy 3D de entrada con valores faltantes (NaNs).

Forma: (n_samples, n_steps, n_features).

Returns:

np.ndarray: Array NumPy 3D con valores faltantes imputados.

`pampaneira_imputation.imputation_methods.impute_median_sample_wise(data)`

Imputa los valores faltantes usando la mediana de cada muestra (a través del tiempo).

Si toda una característica para una muestra es NaN, usa un valor de reserva.

Args:**data (ndarray):** Array NumPy de forma (n_samples, n_timestamps, n_features)

Datos de entrada con valores NaN para ser imputados.

Returns:**ndarray:** Array NumPy de forma (n_samples, n_timestamps, n_features)

Datos con valores NaN imputados.

1.7 pampaneira_imputation.utils module

`pampaneira_imputation.utils.create_missingness(data: ndarray, rate: float, pattern: str, **kwargs) → ndarray`

Introduce valores faltantes (NaN) en un array NumPy 3D (muestras, pasos, características).

Modifica el array in situ por eficiencia, pero también lo devuelve.

Args:**data (np.ndarray):** Array NumPy 3D de entrada. Forma: (n_samples, n_steps, n_features). **rate (float):** Tasa de datos faltantes a introducir (entre 0 y 1). **pattern (str):** Patrón de datos faltantes: 'point', 'subseq' o 'block'. ****kwargs:** Argumentos adicionales dependientes del patrón:

- Para 'subseq': min_missing_len, max_missing_len.
- Para 'block': min_missing_len, max_missing_len.

Returns:

np.ndarray: Array NumPy 3D con valores faltantes introducidos.

`pampaneira_imputation.utils.reshape_imputed_to_df(imputed_data: ndarray, original_index: DatetimeIndex, columns: List[str], n_steps: int) → DataFrame`

Remodela los datos imputados 3D (muestras, pasos, características) de vuelta a un DataFrame 2D.

Asume que el índice original corresponde al *inicio* de cada ventana. Reconstruye la línea de tiempo completa.**Args:****imputed_data (np.ndarray):** Array NumPy 3D de datos imputados.

Forma: (n_samples, n_steps, n_features).

original_index (pd.DatetimeIndex): DatetimeIndex original correspondiente al conjunto de datos a imputar. **columns (List[str]):** Lista de nombres de columnas para el DataFrame reconstruido. **n_steps (int):** Número de pasos de tiempo en cada ventana.**Returns:**

pd.DataFrame: DataFrame de Pandas reconstruido a partir de los datos imputados.

`pampaneira_imputation.utils.sliding_window(data: ndarray, n_steps: int) → ndarray`

Crea ventanas deslizantes a partir de datos secuenciales.

Args:

data (np.ndarray): Array NumPy 2D o 3D de datos secuenciales.

Si es 2D, se asume forma (n_timesteps, n_features). Si es 3D, se asume forma (n_samples, n_timesteps, n_features).

n_steps (int): Tamaño de cada ventana deslizante.

Returns:

np.ndarray: Array NumPy 3D de ventanas deslizantes.

Forma: (n_windows, n_steps, n_features) si la entrada era 2D,

o (n_samples, n_windows, n_steps, n_features) si la entrada era 3D. En este caso, siempre devuelve 3D con forma (n_windows, n_steps, n_features).

1.8 Module contents

Módulo de inicialización para el paquete `pampaneira_imputation`.

TESTS PACKAGE

2.1 Submodules

2.2 tests.conftest module

2.3 tests.minimal_impute_function module

2.4 tests.minimal_test module

2.5 tests.test_data_preprocessor module

2.6 tests.test_evaluation module

2.7 tests.test_function_standalone module

2.8 tests.test_imputation_methods module

2.9 tests.test_utils module

2.10 Module contents

PYTHON MODULE INDEX

p

- `pampaneira_imputation`, [12](#)
- `pampaneira_imputation.config`, [3](#)
- `pampaneira_imputation.data_loader`, [3](#)
- `pampaneira_imputation.data_preprocessor`, [5](#)
- `pampaneira_imputation.evaluation`, [9](#)
- `pampaneira_imputation.imputation_methods`, [9](#)
- `pampaneira_imputation.utils`, [11](#)

INDEX

A

`add_period1_padding()` (in module `pampaneira_imputation.data_preprocessor`), 5

C

`calculate_imputation_metrics()` (in module `pampaneira_imputation.evaluation`), 9
`create_missingness()` (in module `pampaneira_imputation.utils`), 11

E

`evaluate_all_methods()` (in module `pampaneira_imputation.evaluation`), 9

F

`fill_missing_timestamps()` (in module `pampaneira_imputation.data_preprocessor`), 5

I

`impute_forward_backward()` (in module `pampaneira_imputation.imputation_methods`), 9
`impute_linear()` (in module `pampaneira_imputation.imputation_methods`), 10
`impute_mean()` (in module `pampaneira_imputation.imputation_methods`), 10
`impute_mean_sample_wise()` (in module `pampaneira_imputation.imputation_methods`), 10
`impute_median()` (in module `pampaneira_imputation.imputation_methods`), 10
`impute_median_sample_wise()` (in module `pampaneira_imputation.imputation_methods`), 11

L

`load_intersection_data()` (in module `pampaneira_imputation.data_loader`), 3

`load_traffic_data()` (in module `pampaneira_imputation.data_loader`), 3

M

module

`pampaneira_imputation`, 12
`pampaneira_imputation.config`, 3
`pampaneira_imputation.data_loader`, 3
`pampaneira_imputation.data_preprocessor`, 5
`pampaneira_imputation.evaluation`, 9
`pampaneira_imputation.imputation_methods`, 9
`pampaneira_imputation.utils`, 11

P

`pampaneira_imputation`
module, 12
`pampaneira_imputation.config`
module, 3
`pampaneira_imputation.data_loader`
module, 3
`pampaneira_imputation.data_preprocessor`
module, 5
`pampaneira_imputation.evaluation`
module, 9
`pampaneira_imputation.imputation_methods`
module, 9
`pampaneira_imputation.utils`
module, 11
`preprocess_for_imputation()` (in module `pampaneira_imputation.data_preprocessor`), 6

R

`reshape_imputed_to_df()` (in module `pampaneira_imputation.utils`), 11

S

`sliding_window()` (in module `pampaneira_imputation.utils`), 11

`split_by_period()` (*in module pam-*
paneira_imputation.data_preprocessor),
[8](#)